

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Skiing Session 4: Transactions and Stored Logic

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO1
Session	Skiing 4	Date	

Aim

To demonstrate the ACID properties with BEGIN/COMMIT/ROLLBACK/SAVEPOINT, observe MVCC row versions, compare isolation levels and the concurrency anomalies (dirty read, non-repeatable read, phantom read), use advisory locks, and implement PL/pgSQL functions, stored procedures, an audit trigger, views and materialised views for Task Hub.

Program

s4.sql (prerequisite: taskhub_db from Skiing 3)

```
-- Skiing 4: Transactions and Stored Logic (PostgreSQL / PL/pgSQL)
-- Prerequisite: taskhub_db created in Skiing 3
\c taskhub_db

ALTER TABLE projects ADD COLUMN budget_hours NUMERIC(6,1) NOT NULL DEFAULT 100
    CHECK (budget_hours >= 0);
UPDATE projects SET budget_hours = 120 WHERE project_id = 1;
UPDATE projects SET budget_hours = 80 WHERE project_id = 2;
SELECT project_id, name, budget_hours FROM projects ORDER BY project_id;

-- ===== ACID: COMMIT =====
-- Atomicity: both updates are applied together
BEGIN;
UPDATE projects SET budget_hours = budget_hours - 20 WHERE project_id = 1;
UPDATE projects SET budget_hours = budget_hours + 20 WHERE project_id = 2;
COMMIT;
SELECT project_id, budget_hours FROM projects WHERE project_id IN (1, 2) ORDER BY project_id;

-- ===== ACID: ROLLBACK =====
-- Consistency: the CHECK (budget_hours >= 0) fails, so the whole transaction is undone
BEGIN;
UPDATE projects SET budget_hours = budget_hours + 500 WHERE project_id = 2;
UPDATE projects SET budget_hours = budget_hours - 500 WHERE project_id = 1;
ROLLBACK;
SELECT project_id, budget_hours FROM projects WHERE project_id IN (1, 2) ORDER BY project_id;

-- ===== SAVEPOINT =====
BEGIN;
INSERT INTO tasks (project_id, assignee_id, title, status) VALUES (1, 3, 'Write unit tests', 'TODO');
SAVEPOINT after_first_task;
INSERT INTO tasks (project_id, assignee_id, title, status) VALUES (1, 3, 'Wrong task', 'TODO');
ROLLBACK TO SAVEPOINT after_first_task; -- undo only the second insert
COMMIT;
SELECT title FROM tasks WHERE title IN ('Write unit tests', 'Wrong task');

-- ===== MVCC: row versions =====
-- xmin = transaction that created this row version, xmax = transaction that deleted/locked it
SELECT xmin, xmax, project_id, budget_hours FROM projects WHERE project_id = 1;
UPDATE projects SET budget_hours = budget_hours WHERE project_id = 1; -- creates a new version
SELECT xmin, xmax, project_id, budget_hours FROM projects WHERE project_id = 1;

-- ===== Isolation levels =====
SHOW default_transaction_isolation;
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SHOW transaction_isolation;
COMMIT;

-- ===== Advisory locks =====
SELECT pg_try_advisory_lock(42) AS lock_acquired;
SELECT locktype, objid, mode, granted FROM pg_locks WHERE locktype = 'advisory';
SELECT pg_advisory_unlock(42) AS lock_released;
```

```

-- ===== Function =====
CREATE OR REPLACE FUNCTION project_completion(p_project_id INT)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    v_total INT;
    v_done INT;
BEGIN
    SELECT COUNT(*), COUNT(*) FILTER (WHERE status = 'DONE')
    INTO v_total, v_done
    FROM tasks WHERE project_id = p_project_id;

    IF v_total = 0 THEN
        RETURN 0;
    END IF;
    RETURN ROUND(100.0 * v_done / v_total, 1);
END;
$$;

SELECT name, project_completion(project_id) AS percent_done FROM projects ORDER BY project_id;

-- ===== Stored procedure (multi-step operation in one transaction) =====
CREATE OR REPLACE PROCEDURE transfer_hours(p_from INT, p_to INT, p_hours NUMERIC)
LANGUAGE plpgsql
AS $$
DECLARE
    v_available NUMERIC;
BEGIN
    IF p_hours <= 0 THEN
        RAISE EXCEPTION 'Hours to transfer must be positive, got %', p_hours;
    END IF;

    SELECT budget_hours INTO v_available FROM projects WHERE project_id = p_from FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Project % does not exist', p_from;
    END IF;
    IF v_available < p_hours THEN
        RAISE EXCEPTION 'Insufficient budget: project % has % hours, requested %',
            p_from, v_available, p_hours;
    END IF;

    UPDATE projects SET budget_hours = budget_hours - p_hours WHERE project_id = p_from;
    UPDATE projects SET budget_hours = budget_hours + p_hours WHERE project_id = p_to;
    RAISE NOTICE 'Transferred % hours from project % to project %', p_hours, p_from, p_to;
END;
$$;

CALL transfer_hours(1, 3, 15);
CALL transfer_hours(2, 1, 1000);
SELECT project_id, name, budget_hours FROM projects ORDER BY project_id;

-- ===== Audit trigger on a critical table =====
CREATE TABLE task_audit (
    audit_id SERIAL PRIMARY KEY,
    task_id INT,
    operation VARCHAR(6) NOT NULL,
    old_status VARCHAR(12),
    new_status VARCHAR(12),
    changed_by TEXT NOT NULL DEFAULT current_user,
    changed_at TIMESTAMP NOT NULL DEFAULT now()
);

CREATE OR REPLACE FUNCTION log_task_change()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO task_audit (task_id, operation, new_status) VALUES (NEW.task_id, TG_OP, NEW.status);
        RETURN NEW;
    ELSIF TG_OP = 'UPDATE' THEN
        IF NEW.status IS DISTINCT FROM OLD.status THEN
            INSERT INTO task_audit (task_id, operation, old_status, new_status)
            VALUES (NEW.task_id, TG_OP, OLD.status, NEW.status);
        END IF;
        RETURN NEW;
    ELSE
        RETURN NEW;
    END IF;
END;

```

```

        INSERT INTO task_audit (task_id, operation, old_status) VALUES (OLD.task_id, TG_OP, OLD.status);
        RETURN OLD;
    END IF;
END;
$$;

CREATE TRIGGER trg_task_audit
AFTER INSERT OR UPDATE OR DELETE ON tasks
FOR EACH ROW EXECUTE FUNCTION log_task_change();

INSERT INTO tasks (project_id, assignee_id, title, status) VALUES (2, 4, 'Accessibility review', 'TODO');
UPDATE tasks SET status = 'IN_PROGRESS' WHERE title = 'Accessibility review';
UPDATE tasks SET status = 'DONE' WHERE title = 'Accessibility review';
DELETE FROM tasks WHERE title = 'Accessibility review';
SELECT audit_id, operation, old_status, new_status, changed_by FROM task_audit ORDER BY audit_id;

-- ===== View =====
CREATE OR REPLACE VIEW open_tasks_view AS
SELECT t.task_id, t.title, t.status, u.full_name AS assignee, t.due_date
FROM   tasks t LEFT JOIN users u ON u.user_id = t.assignee_id
WHERE  t.status <> 'DONE';

SELECT * FROM open_tasks_view ORDER BY due_date;

-- ===== Materialized view =====
CREATE MATERIALIZED VIEW team_hours_mv AS
SELECT u.team, SUM(t.hours_logged) AS total_hours, COUNT(*) AS tasks
FROM   tasks t JOIN users u ON u.user_id = t.assignee_id
GROUP BY u.team;

SELECT * FROM team_hours_mv ORDER BY team;
UPDATE tasks SET hours_logged = hours_logged + 10 WHERE title = 'Write FastAPI CRUD';
SELECT * FROM team_hours_mv ORDER BY team; -- still the old snapshot
REFRESH MATERIALIZED VIEW team_hours_mv;
SELECT * FROM team_hours_mv ORDER BY team; -- refreshed

```

concurrency_demo.py (two concurrent sessions A and B)

```

"""
Skilling 4 (Part B): Concurrency anomalies and isolation levels in PostgreSQL.
Two independent connections (session A and session B) act like two users.

Run: pip install psycopg2-binary
     python concurrency_demo.py
"""
import os
import psycopg2
from psycopg2 import errors
from psycopg2.extensions import (ISOLATION_LEVEL_READ_COMMITTED,
                                ISOLATION_LEVEL_READ_UNCOMMITTED,
                                ISOLATION_LEVEL_REPEATABLE_READ,
                                ISOLATION_LEVEL_SERIALIZABLE)

DSN = os.getenv("TASKHUB_DSN", "dbname=taskhub_db user=postgres password=postgres host=localhost")

def connect(level):
    conn = psycopg2.connect(DSN)
    conn.set_isolation_level(level)
    return conn

def scalar(cur, sql, params=None):
    cur.execute(sql, params)
    return cur.fetchone()[0]

def reset():
    with psycopg2.connect(DSN) as conn, conn.cursor() as cur:
        cur.execute("UPDATE projects SET budget_hours = 100 WHERE project_id = 1")
        cur.execute("DELETE FROM tasks WHERE title LIKE 'Phantom%'")

BUDGET = "SELECT budget_hours FROM projects WHERE project_id = 1"
COUNT_TODO = "SELECT COUNT(*) FROM tasks WHERE project_id = 1 AND status = 'TODO'"

def dirty_read():
    print("1. DIRTY READ (READ UNCOMMITTED requested)")

```

```

    reset()
    a, b = connect(ISOLATION_LEVEL_READ_UNCOMMITTED), connect(ISOLATION_LEVEL_READ_UNCOMMITTED)
    ca, cb = a.cursor(), b.cursor()
    cb.execute("UPDATE projects SET budget_hours = 999 WHERE project_id = 1")    # not committed
    print(f"    B updated budget to 999 (uncommitted)")
    print(f"    A reads budget = {scalar(ca, BUDGET)} -> dirty value NOT visible")
    b.rollback(); a.rollback(); a.close(); b.close()
    print("    PostgreSQL treats READ UNCOMMITTED as READ COMMITTED, so dirty reads never happen.\n")

def non_repeatable_read(level, name):
    print(f"2. NON-REPEATABLE READ under {name}")
    reset()
    a, b = connect(level), connect(ISOLATION_LEVEL_READ_COMMITTED)
    ca, cb = a.cursor(), b.cursor()
    first = scalar(ca, BUDGET)
    cb.execute("UPDATE projects SET budget_hours = 60 WHERE project_id = 1"); b.commit()
    second = scalar(ca, BUDGET)
    print(f"    A first read = {first}, B commits 60, A second read = {second}"
          f"    -> {'ANOMALY (value changed)' if first != second else 'prevented (same snapshot)'}")
    a.commit(); a.close(); b.close()
    print()

def phantom_read(level, name):
    print(f"3. PHANTOM READ under {name}")
    reset()
    a, b = connect(level), connect(ISOLATION_LEVEL_READ_COMMITTED)
    ca, cb = a.cursor(), b.cursor()
    first = scalar(ca, COUNT_TODO)
    cb.execute("INSERT INTO tasks (project_id, title, status) VALUES (1, 'Phantom task', 'TODO')")
    b.commit()
    second = scalar(ca, COUNT_TODO)
    print(f"    A counts TODO tasks = {first}, B inserts one, A counts again = {second}"
          f"    -> {'ANOMALY (new row appeared)' if first != second else 'prevented'}")
    a.commit(); a.close(); b.close()
    print()

def lost_update_serializable():
    print("4. CONCURRENT UPDATE under SERIALIZABLE (lost update prevention)")
    reset()
    a, b = connect(ISOLATION_LEVEL_SERIALIZABLE), connect(ISOLATION_LEVEL_SERIALIZABLE)
    ca, cb = a.cursor(), b.cursor()
    va, vb = scalar(ca, BUDGET), scalar(cb, BUDGET)
    ca.execute("UPDATE projects SET budget_hours = %s WHERE project_id = 1", (va - 10,)); a.commit()
    print(f"    A read {va}, wrote {va - 10} and committed")
    try:
        cb.execute("UPDATE projects SET budget_hours = %s WHERE project_id = 1", (vb - 30,))
        b.commit()
        print("    B committed (lost update)")
    except errors.SerializationFailure as exc:
        b.rollback()
        print(f"    B read {vb}, tried to write {vb - 30} -> {type(exc).__name__}: "
              f"{exc.pgerror.splitlines()[0]}")
        print("    B must retry the transaction; A's update is not lost.")
    with psycopg2.connect(DSN) as c, c.cursor() as cur:
        print(f"    Final budget = {scalar(cur, BUDGET)}")
    a.close(); b.close()
    print()

def advisory_lock_demo():
    print("5. ADVISORY LOCK (application-level mutual exclusion)")
    a, b = connect(ISOLATION_LEVEL_READ_COMMITTED), connect(ISOLATION_LEVEL_READ_COMMITTED)
    ca, cb = a.cursor(), b.cursor()
    print(f"    A pg_try_advisory_lock(1001) -> {scalar(ca, 'SELECT pg_try_advisory_lock(1001)')}")
    print(f"    B pg_try_advisory_lock(1001) -> {scalar(cb, 'SELECT pg_try_advisory_lock(1001)')}")
    print("    (blocked: A holds it)")
    scalar(ca, "SELECT pg_advisory_unlock(1001)")
    print(f"    A unlocks; B tries again -> {scalar(cb, 'SELECT pg_try_advisory_lock(1001)')}")
    scalar(cb, "SELECT pg_advisory_unlock(1001)")
    a.close(); b.close()

if __name__ == "__main__":
    dirty_read()
    non_repeatable_read(ISOLATION_LEVEL_READ_COMMITTED, "READ COMMITTED")

```

```

non_repeatable_read(ISOLATION_LEVEL_REPEATABLE_READ, "REPEATABLE READ")
phantom_read(ISOLATION_LEVEL_READ_COMMITTED, "READ COMMITTED")
phantom_read(ISOLATION_LEVEL_REPEATABLE_READ, "REPEATABLE READ")
lost_update_serializable()
advisory_lock_demo()
reset()

```

Output

psql -f s4.sql

```

ALTER TABLE projects ADD COLUMN budget_hours NUMERIC(6,1) NOT NULL DEFAULT 100
CHECK (budget_hours >= 0);
ALTER TABLE
UPDATE projects SET budget_hours = 120 WHERE project_id = 1;
UPDATE 1
UPDATE projects SET budget_hours = 80 WHERE project_id = 2;
UPDATE 1
SELECT project_id, name, budget_hours FROM projects ORDER BY project_id;
 project_id |      name      | budget_hours
-----+-----+-----
          1 | Task Hub Backend |         120.0
          2 | Task Hub Frontend |          80.0
          3 | Mobile App      |         100.0
(3 rows)

BEGIN;
BEGIN
UPDATE projects SET budget_hours = budget_hours - 20 WHERE project_id = 1;
UPDATE 1
UPDATE projects SET budget_hours = budget_hours + 20 WHERE project_id = 2;
UPDATE 1
COMMIT;
COMMIT
SELECT project_id, budget_hours FROM projects WHERE project_id IN (1, 2) ORDER BY project_id;
 project_id | budget_hours
-----+-----
          1 |         100.0
          2 |         100.0
(2 rows)

BEGIN;
BEGIN
UPDATE projects SET budget_hours = budget_hours + 500 WHERE project_id = 2;
UPDATE 1
UPDATE projects SET budget_hours = budget_hours - 500 WHERE project_id = 1;
ERROR:  new row for relation "projects" violates check constraint "projects_budget_hours_check"
DETAIL:  Failing row contains (1, Task Hub Backend, 1, -400.0).
ROLLBACK;
ROLLBACK
SELECT project_id, budget_hours FROM projects WHERE project_id IN (1, 2) ORDER BY project_id;
 project_id | budget_hours
-----+-----
          1 |         100.0
          2 |         100.0
(2 rows)

BEGIN;
BEGIN
INSERT INTO tasks (project_id, assignee_id, title, status) VALUES (1, 3, 'Write unit tests', 'TODO');
INSERT 0 1
SAVEPOINT after_first_task;
SAVEPOINT
INSERT INTO tasks (project_id, assignee_id, title, status) VALUES (1, 3, 'Wrong task', 'TODO');
INSERT 0 1
ROLLBACK TO SAVEPOINT after_first_task;
ROLLBACK
COMMIT;
COMMIT
SELECT title FROM tasks WHERE title IN ('Write unit tests', 'Wrong task');
 title
-----
Write unit tests
(1 row)

SELECT xmin, xmax, project_id, budget_hours FROM projects WHERE project_id = 1;
 xmin | xmax | project_id | budget_hours
-----+-----+-----+-----
 1034 | 1036 |          1 |         100.0

```

```

(1 row)

UPDATE projects SET budget_hours = budget_hours WHERE project_id = 1;
UPDATE 1
SELECT xmin, xmax, project_id, budget_hours FROM projects WHERE project_id = 1;
  xmin | xmax | project_id | budget_hours
-----+-----+-----+-----
  1038 |    0 |          1 |         100.0
(1 row)

SHOW default_transaction_isolation;
 default_transaction_isolation
-----
 read committed
(1 row)

BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN
SHOW transaction_isolation;
 transaction_isolation
-----
 repeatable read
(1 row)

COMMIT;
COMMIT
SELECT pg_try_advisory_lock(42) AS lock_acquired;
 lock_acquired
-----
 t
(1 row)

SELECT locktype, objid, mode, granted FROM pg_locks WHERE locktype = 'advisory';
 locktype | objid | mode | granted
-----+-----+-----+-----
 advisory |    42 | ExclusiveLock | t
(1 row)

SELECT pg_advisory_unlock(42) AS lock_released;
 lock_released
-----
 t
(1 row)

CREATE OR REPLACE FUNCTION project_completion(p_project_id INT)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    v_total INT;
    v_done INT;
BEGIN
    SELECT COUNT(*), COUNT(*) FILTER (WHERE status = 'DONE')
    INTO v_total, v_done
    FROM tasks WHERE project_id = p_project_id;

    IF v_total = 0 THEN
        RETURN 0;
    END IF;
    RETURN ROUND(100.0 * v_done / v_total, 1);
END;
$$;
CREATE FUNCTION
SELECT name, project_completion(project_id) AS percent_done FROM projects ORDER BY project_id;
  name | percent_done
-----+-----
 Task Hub Backend |         33.3
 Task Hub Frontend |          0.0
  Mobile App |          0
(3 rows)

CREATE OR REPLACE PROCEDURE transfer_hours(p_from INT, p_to INT, p_hours NUMERIC)
LANGUAGE plpgsql
AS $$
DECLARE
    v_available NUMERIC;
BEGIN
    IF p_hours <= 0 THEN

```

```

        RAISE EXCEPTION 'Hours to transfer must be positive, got %', p_hours;
    END IF;

    SELECT budget_hours INTO v_available FROM projects WHERE project_id = p_from FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Project % does not exist', p_from;
    END IF;
    IF v_available < p_hours THEN
        RAISE EXCEPTION 'Insufficient budget: project % has % hours, requested %',
            p_from, v_available, p_hours;
    END IF;

    UPDATE projects SET budget_hours = budget_hours - p_hours WHERE project_id = p_from;
    UPDATE projects SET budget_hours = budget_hours + p_hours WHERE project_id = p_to;
    RAISE NOTICE 'Transferred % hours from project % to project %', p_hours, p_from, p_to;
END;
$$;
CREATE PROCEDURE
CALL transfer_hours(1, 3, 15);
NOTICE: Transferred 15 hours from project 1 to project 3
CALL
CALL transfer_hours(2, 1, 1000);
ERROR: Insufficient budget: project 2 has 100.0 hours, requested 1000
CONTEXT: PL/pgSQL function transfer_hours(integer,integer,numeric) line 14 at RAISE
SELECT project_id, name, budget_hours FROM projects ORDER BY project_id;
 project_id |      name      | budget_hours
-----+-----+-----
          1 | Task Hub Backend |         85.0
          2 | Task Hub Frontend |        100.0
          3 | Mobile App      |        115.0
(3 rows)

CREATE TABLE task_audit (
    audit_id SERIAL PRIMARY KEY,
    task_id INT,
    operation VARCHAR(6) NOT NULL,
    old_status VARCHAR(12),
    new_status VARCHAR(12),
    changed_by TEXT NOT NULL DEFAULT current_user,
    changed_at TIMESTAMP NOT NULL DEFAULT now()
);
CREATE TABLE
CREATE OR REPLACE FUNCTION log_task_change()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO task_audit (task_id, operation, new_status) VALUES (NEW.task_id, TG_OP, NEW.status);
        RETURN NEW;
    ELSIF TG_OP = 'UPDATE' THEN
        IF NEW.status IS DISTINCT FROM OLD.status THEN
            INSERT INTO task_audit (task_id, operation, old_status, new_status)
            VALUES (NEW.task_id, TG_OP, OLD.status, NEW.status);
        END IF;
        RETURN NEW;
    ELSE
        INSERT INTO task_audit (task_id, operation, old_status) VALUES (OLD.task_id, TG_OP, OLD.status);
        RETURN OLD;
    END IF;
END;
$$;
CREATE FUNCTION
CREATE TRIGGER trg_task_audit
AFTER INSERT OR UPDATE OR DELETE ON tasks
FOR EACH ROW EXECUTE FUNCTION log_task_change();
CREATE TRIGGER
INSERT INTO tasks (project_id, assignee_id, title, status) VALUES (2, 4, 'Accessibility review', 'TODO');
INSERT 0 1
UPDATE tasks SET status = 'IN_PROGRESS' WHERE title = 'Accessibility review';
UPDATE 1
UPDATE tasks SET status = 'DONE' WHERE title = 'Accessibility review';
UPDATE 1
DELETE FROM tasks WHERE title = 'Accessibility review';
DELETE 1
SELECT audit_id, operation, old_status, new_status, changed_by FROM task_audit ORDER BY audit_id;
 audit_id | operation | old_status | new_status | changed_by
-----+-----+-----+-----+-----

```

```

1 | INSERT | | TODO | postgres
2 | UPDATE | TODO | IN_PROGRESS | postgres
3 | UPDATE | IN_PROGRESS | DONE | postgres
4 | DELETE | DONE | | postgres
(4 rows)

```

```

CREATE OR REPLACE VIEW open_tasks_view AS
SELECT t.task_id, t.title, t.status, u.full_name AS assignee, t.due_date
FROM tasks t LEFT JOIN users u ON u.user_id = t.assignee_id
WHERE t.status <> 'DONE';

```

```
CREATE VIEW
```

```
SELECT * FROM open_tasks_view ORDER BY due_date;
```

task_id	title	status	assignee	due_date
5	Build dashboard page	IN_PROGRESS	Ananya Rao	2026-10-05
3	Write FastAPI CRUD	IN_PROGRESS	Arjun Nair	2026-10-08
6	Replace mock data	TODO	Sneha Reddy	2026-10-12
4	Add JWT authentication	IN_PROGRESS	Rahul Verma	2026-10-15
8	Set up CI pipeline	TODO		2026-10-20
9	Write unit tests	TODO	Rahul Verma	

```
(6 rows)
```

```

CREATE MATERIALIZED VIEW team_hours_mv AS
SELECT u.team, SUM(t.hours_logged) AS total_hours, COUNT(*) AS tasks
FROM tasks t JOIN users u ON u.user_id = t.assignee_id
GROUP BY u.team;

```

```
SELECT 2
```

```
SELECT * FROM team_hours_mv ORDER BY team;
```

team	total_hours	tasks
Backend	28.5	5
Frontend	10.5	2

```
(2 rows)
```

```
UPDATE tasks SET hours_logged = hours_logged + 10 WHERE title = 'Write FastAPI CRUD';
```

```
UPDATE 1
```

```
SELECT * FROM team_hours_mv ORDER BY team;
```

team	total_hours	tasks
Backend	28.5	5
Frontend	10.5	2

```
(2 rows)
```

```
REFRESH MATERIALIZED VIEW team_hours_mv;
```

```
REFRESH MATERIALIZED VIEW
```

```
SELECT * FROM team_hours_mv ORDER BY team;
```

team	total_hours	tasks
Backend	38.5	5
Frontend	10.5	2

```
(2 rows)
```

python concurrency_demo.py

1. DIRTY READ (READ UNCOMMITTED requested)
B updated budget to 999 (uncommitted)
A reads budget = 100.0 -> dirty value NOT visible
PostgreSQL treats READ UNCOMMITTED as READ COMMITTED, so dirty reads never happen.
2. NON-REPEATABLE READ under READ COMMITTED
A first read = 100.0, B commits 60, A second read = 60.0 -> ANOMALY (value changed)
2. NON-REPEATABLE READ under REPEATABLE READ
A first read = 100.0, B commits 60, A second read = 100.0 -> prevented (same snapshot)
3. PHANTOM READ under READ COMMITTED
A counts TODO tasks = 2, B inserts one, A counts again = 3 -> ANOMALY (new row appeared)
3. PHANTOM READ under REPEATABLE READ
A counts TODO tasks = 2, B inserts one, A counts again = 2 -> prevented
4. CONCURRENT UPDATE under SERIALIZABLE (lost update prevention)
A read 100.0, wrote 90.0 and committed
B read 100.0, tried to write 70.0 -> SerializationFailure: ERROR: could not serialize access due to concurrent update
B must retry the transaction; A's update is not lost.
Final budget = 90.0
5. ADVISORY LOCK (application-level mutual exclusion)

```
A pg_try_advisory_lock(1001) -> True  
B pg_try_advisory_lock(1001) -> False (blocked: A holds it)  
A unlocks; B tries again -> True
```

Result

The committed transfer was applied atomically, the transaction that violated the CHECK constraint was rolled back completely, and ROLLBACK TO SAVEPOINT undid only the second insert. xmin/xmax showed that an UPDATE creates a new row version (MVCC). Dirty reads never occur in PostgreSQL; non-repeatable and phantom reads occur under READ COMMITTED and are prevented under REPEATABLE READ; SERIALIZABLE raised a serialization failure instead of losing an update. The procedure transfer_hours, the function project_completion, the audit trigger and the materialised view (stale until REFRESH) worked as intended.